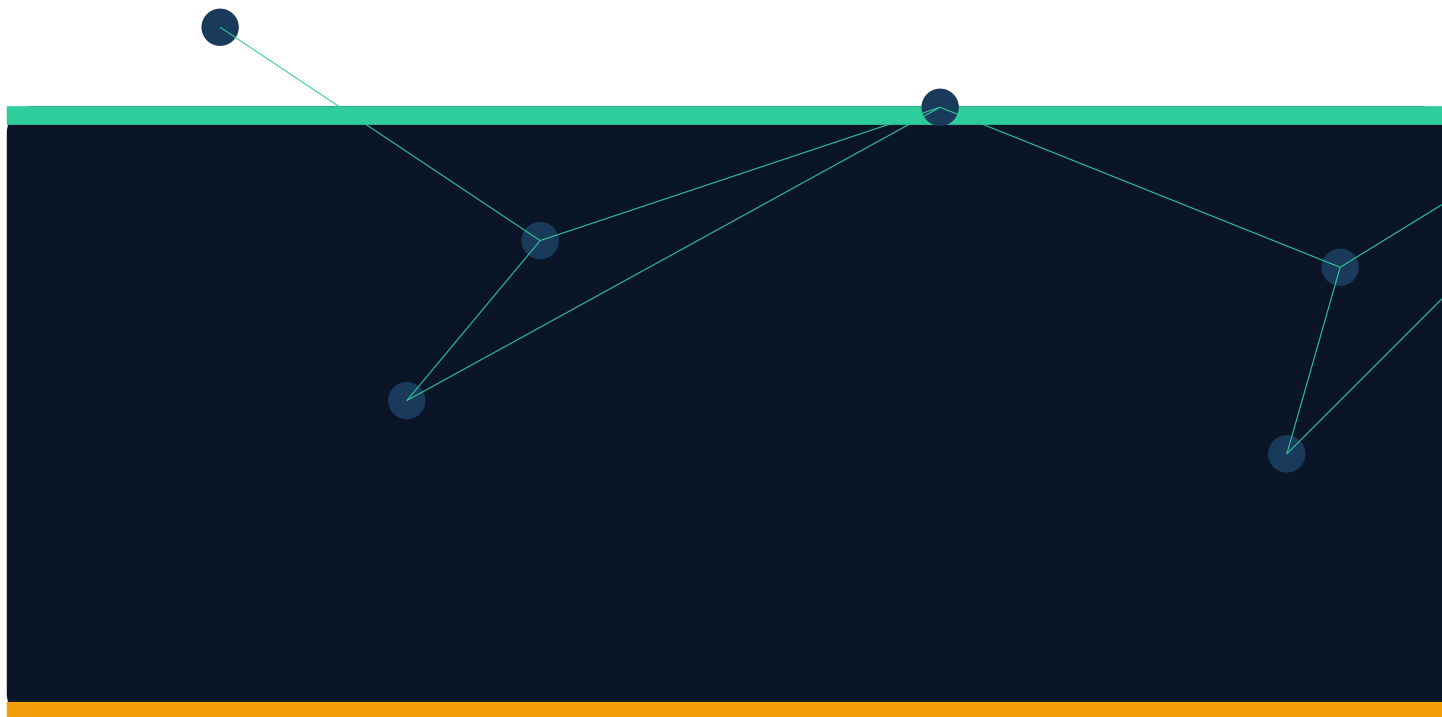




Week 6 — Model Context Protocol (MCP) A Transformative Framework for AI Agents

What You Will Learn

What MCP is & why it matters • The USB-C analogy • Three MCP components • Transport mechanisms (Stdio vs SSE) • Distributed Agent Runtime • MCP vs direct tools • Building an MCP server • Connecting agents to MCP • Framework selection & key engineering principles

SECTION 1 — THEORY

1. What is the Model Context Protocol (MCP)?

The **Model Context Protocol (MCP)** is an open standard — published by Anthropic in November 2024 — that defines how AI agents communicate with external tools, data sources, and services. Think of it as the communication backbone of the modern agentic AI ecosystem.

A crucial clarification upfront: **MCP is not a new type of agent, nor does it fundamentally change how agents reason or act.** It is a **protocol** — a standardised set of rules and message formats for how an AI host (e.g., Claude, GPT-4) connects to and communicates with external capability servers. Think of it exactly like HTTP for the web: HTTP didn't invent the internet, it just gave every device a common language to exchange information reliably.

1.1 The USB-C Analogy

Before MCP, every AI application that wanted to connect to a tool — say, a web search engine, a database, or a code interpreter — had to write its own bespoke integration. This created a massive fragmentation problem: $N \text{ agents} \times M \text{ tools} = N \times M$ custom integrations to build and maintain.

Before MCP (Fragmented)	After MCP (Standardised)
Each agent needs custom code to connect to each tool. Claude connector for Slack $\neq$ GPT-4 connector for Slack. Every new tool = new integration work for every agent.	Any MCP-compatible agent connects to any MCP server automatically. Build one MCP server for Slack $\rightarrow$ works with Claude, GPT-4, Gemini, etc. New tool = build one MCP server, all agents instantly gain access.
$N \text{ agents} \times M \text{ tools} = N \times M \text{ integrations}$ ■	$N \text{ agents} + M \text{ tools} = N + M \text{ components}$ ■

■ IMPORTANT

MCP is to AI agents what USB-C is to devices: a universal port. Before USB-C, every device needed different chargers. After USB-C, one cable works everywhere. MCP does the same for agent-tool connectivity.

2. The Three Core Components of MCP

Every MCP-based system is composed of three distinct roles. Understanding how they interact is the foundation for building any MCP-powered application.

Component	Role	Examples	Analogy
01 Host (LLM App)	The AI application that the user interacts with. It contains the MCP Client and orchestrates the overall workflow.	Claude Desktop, ChatGPT, custom AI chatbot, OpenAI Agents SDK app	The laptop/phone (the device with a port)
02 Client (inside Host)	Lives inside the Host. Manages the connection to one or more MCP Servers, discovers available tools, and relays tool calls/results.	MCPServerStdio, MCPServerSse classes in the Agents SDK	The USB-C port on the laptop
03 MCP Server	A standalone process (local or remote) that exposes Tools, Resources, and Prompts to any MCP Client that connects to it.	Fetch-MCP (web browsing), Playwright MCP (browser automation), filesystem MCP, Slack MCP, custom business logic server	The USB charger / peripheral device

2.1 What Does an MCP Server Provide?

An MCP Server is not just a tool registry. It can expose three distinct types of capabilities to any connecting client:

Tools	Executable functions the agent can call (like API calls, computations, file operations). The LLM decides when to call them based on the task. <i>Example: e.g., <code>search_web()</code>, <code>send_email()</code>, <code>run_sql_query()</code></i>
-------	---

Resources	Contextual data the agent can read — documents, database records, file contents, API responses. Resources enrich the agent's context without executing code. <i>Example: e.g., company_policy.pdf, customer_record:42, git_diff</i>
Prompts	Reusable prompt templates that the server exposes. The host can inject these into the conversation to guide agent behaviour in a standardised way. <i>Example: e.g., 'summarise_ticket' prompt, 'code_review' template</i>

3. Transport Mechanisms — How Client & Server Talk

MCP clients and servers need a communication channel. The protocol supports two distinct transport mechanisms, each suited to different deployment scenarios. Understanding these is essential for architecting your MCP setup correctly.

Dimension	Stdio (Standard I/O)	SSE (Server-Sent Events)
Full Name	Standard Input/Output	Server-Sent Events (HTTP streaming)
How it works	Host spawns MCP server as a child process. Communication happens via stdin/stdout pipes — the host writes JSON, server reads it and responds.	Server runs as a standalone HTTP server. Client connects via HTTP and receives events asynchronously over a persistent connection.
Where server runs	Same machine as the host (local process)	Can be on a remote server, in the cloud, or locally
Best for	Local tools: filesystem, local code execution, local databases	Remote services: cloud APIs, web browsing, shared MCP servers
SDK Class	<code>MCPServerStdio</code>	<code>MCPServerSse</code>
Example	Running Playwright MCP locally to control a browser on your machine	Connecting to a hosted Slack MCP server via HTTPS

■ TIP

In most real-world deployments, MCP servers run outside the host machine (on remote servers). Your client still connects to them — it just uses SSE over HTTP instead of stdin/stdout pipes. The MCP server can take advantage of server-side functionality like internet access, GPU compute, or persistent databases.

4. Distributed Agent Runtime

As agentic systems grow, a single process running all agents on one machine becomes a bottleneck. The **Distributed Agent Runtime** pattern solves this by breaking agent execution across multiple processes and machines — while MCP provides the glue that holds it all together.

Component	Responsibility	Key Behaviour
-----------	----------------	---------------

Host Service	Central coordinator of the distributed system. Receives messages from users/orchestrators and routes them to the appropriate Worker Runtime.	Handles message delivery, maintains agent registry, manages sensors for direct inter-agent messaging.
Worker Runtime	An independent process (potentially on a separate machine) that runs one or more specialised agents. Advertises its agents to the Host Service.	Accepts tasks from the Host, executes agent logic, reports results back. Multiple workers enable true parallelism.
Gremlin (HTTP Manager)	A local HTTP management server that controls the lifecycle of worker processes on a given machine.	Starts/stops workers, monitors health, exposes management APIs for the orchestration layer.

■ IMPORTANT

The Distributed Agent Runtime is the production-scale evolution of the single-process multi-agent pattern from Week 2. MCP makes it possible: each Worker Runtime is itself an MCP server, and the Host Service is the MCP client that orchestrates them all.

5. MCP Servers vs. Direct Tool Functions

You already know how to give agents tools using the `@tool` decorator in the Agents SDK. When should you use that approach vs. connecting to an MCP server? This is one of the most common architectural decisions you will face.

Dimension	Direct <code>@tool</code> Functions	MCP Server
Definition	Python function decorated with <code>@tool</code> , living in the same codebase as the agent	Independent process/service exposing tools via MCP protocol
Reusability	Tied to one agent application — must be reimplemented elsewhere	Any MCP-compatible agent or host can connect to it once built
Deployment	Runs in the same process as the agent	Runs independently — can be on any machine, scaled separately
Discovery	Manually configured in the agent's <code>tools=[]</code> list	Agent auto-discovers all tools on connection to the server
Best for	Simple, single-use tools; rapid prototyping; team-specific logic	Shared capabilities; production services; cross-team tool libraries
Example	A custom function to format a report in your company's template	A shared Playwright MCP server used by 10 different agent apps

6. Choosing Your Agentic Framework

The agentic AI ecosystem now has many competing frameworks: OpenAI Agents SDK, LangChain, LangGraph, Google ADK, AutoGen, CrewAI, and more. The critical insight is: **the most important question is not 'which framework is best?' but 'which framework fits my team's skills and my problem's**

structure?'

All mature frameworks share common foundations (LLM calls, tool use, memory, multi-agent orchestration), and all are increasingly MCP-compatible. The real differentiators are developer experience, community support, and alignment with your team's existing technology stack.

Framework	Best For	Strength	MCP Support
OpenAI Agents SDK	Multi-agent pipelines, production OpenAI deployments	Clean async API, native handoffs, guardrails	Native (MCPServerStdio, SSE)
LangGraph	Complex stateful workflows, cyclic agent graphs	Visual graph builder, fine-grained state control	Via LangChain tool adapters
AutoGen / AG2	Conversational multi-agent systems, research	Role-playing agents, group chat dynamics	Experimental support
CrewAI	Task-based crew pipelines, simpler workflows	Beginner-friendly, role/goal abstractions	Community adapters
Google ADK	Google Cloud deployments, Gemini-native apps	Deep Gemini integration, BigQuery/GCS tools	Native (Google's MCP-compatible layer)

7. Key Engineering Principles for Agentic AI

Building reliable, production-grade agentic systems requires disciplined engineering habits. These four principles — distilled from practitioners who have shipped real agentic systems at scale — are the difference between a demo that works once and a system that works every time.

1	Metric for Success Define a quantitative success metric before you write a single line of agent code. What does 'working correctly' mean for your pipeline? Precision/recall for a classification agent? Email reply rate for an SDR agent? Cost per completed task? Without a metric, you cannot tell if your changes are improvements or regressions. This forces you to think about evaluation early — which is the single biggest predictor of production success in agentic AI.
2	Workflow Over Autonomy Start with a deterministic, hardcoded workflow before introducing agent autonomy. An orchestrator that always calls Researcher → Writer → Sender in a fixed sequence is more reliable, debuggable, and explainable than one that decides the sequence autonomously. Once the fixed workflow works reliably, gradually introduce decision points where the agent chooses the path. Never start fully autonomous.
3	Work Bottom-Up Build the smallest possible working unit first — a single tool, a single agent, one handoff — and verify it works before adding the next piece. This approach, sometimes called 'unit testing for agents', prevents the common failure mode where a complex 10-agent system breaks mysteriously and you have no idea which agent or which tool is the culprit. Build incrementally, test at every step.

Context Over Memory

LLMs perform dramatically better when given the right context in their prompt than when relying on external memory retrieval. Before reaching for vector databases and RAG pipelines, ask: 'Can I simply pass the relevant information directly in the system prompt or as a resource?' In many cases, well-structured context injected at the right moment outperforms complex memory architectures. Memory adds latency, complexity, and failure modes — use it only when necessary.

8. Problem Solving & Debugging Agentic Systems

Debugging agentic systems is fundamentally different from debugging traditional software. The non-determinism of LLM outputs, the asynchronous nature of multi-agent flows, and the complexity of tool call chains make conventional print-debugging insufficient. Here is the systematic approach used by experienced agentic AI engineers:

Strategy	What to Do	Why It Works
Give Better Context	Rewrite the agent's system prompt. Add examples of correct behaviour. Specify the output format explicitly. Add constraints.	80% of agent failures are prompt failures, not code failures. Better context = better LLM decisions.
Read the Traces	Use the SDK's built-in tracing or OpenTelemetry to capture the full agent execution: every LLM call, every tool invocation, every handoff, every token of input and output.	You cannot debug what you cannot observe. Traces reveal exactly where the agent's reasoning diverged from expected.
Be a Scientist	Form a specific hypothesis ('Agent fails because tool X returns too much data'). Test it in isolation. Measure the result. Iterate.	Random 'let me try changing this and see' debugging wastes hours. Scientific method + evals = systematic improvement.
Isolate Components	Test each tool, each agent, each handoff independently before testing the full pipeline. Use hardcoded inputs to isolate variables.	In complex multi-agent systems, failures cascade. Isolation identifies the root cause before it propagates.

SECTION 2 — PRACTICAL & CODING

The following cells cover the full MCP development workflow: building an MCP server from scratch, connecting it to an agent, and integrating a pre-built MCP server (Playwright) into a real agent pipeline.

Cell [1]

Environment Setup & Installation

Purpose

Install all required packages and configure the environment for MCP development.

Cell [1] — Environment Setup

```
# Install required packages
# Run in terminal (not notebook cell)

# Core agent SDK with MCP support
# pip install openai-agents python-dotenv

# MCP Python SDK (for building MCP servers)
# pip install mcp

# For Playwright MCP server (browser automation)
# npm install -g @playwright/mcp
# npx playwright install # download browser binaries

# Verify installations

import asyncio
from dotenv import load_dotenv
from agents import Agent, Runner
from agents.mcp import MCPServerStdio, MCPServerSse

load_dotenv(override=True)
print('Environment ready for MCP development!')
```

■ OUTPUT / CONSOLE

```
Environment ready for MCP development!
```

Line-by-Line Breakdown

pip install mcp — Installs Anthropic's Python MCP SDK, which provides the server-side framework for building MCP servers. This gives you the `FastMCP` class, the `@server.tool()` decorator, and all MCP protocol primitives.

pip install openai-agents — Installs the OpenAI Agents SDK which includes built-in MCP client support via `MCPServerStdio` and `MCPServerSse` classes. These are the bridge between your agents and any MCP server.

`npm install -g @playwright/mcp` — Installs the Playwright MCP server as a Node.js package. This is a pre-built, production-ready MCP server that exposes browser automation capabilities (click, type, navigate, screenshot) to any MCP-compatible agent. It runs as a local process (Stdio transport).

`from agents.mcp import MCPServerStdio, MCPServerSse` — Imports the two MCP client classes. `MCPServerStdio` is for local process-based MCP servers. `MCPServerSse` is for remote HTTP-based servers. Both implement the same interface from the agent's perspective.

■ **Key Insight:** Notice that Playwright MCP is a Node.js package, not Python. MCP is language-agnostic — any language can implement an MCP server as long as it speaks the protocol. Your Python agents can connect to JS, Go, or Rust MCP servers.

Purpose

Create a custom MCP server that exposes two tools — a company info lookup and a CRM record fetcher — demonstrating the server-side MCP development pattern.

Cell [2] — Building an MCP Server

```
# File: my_mcp_server.py

# Run this as a standalone process: python my_mcp_server.py

from mcp.server.fastmcp import FastMCP
from typing import Optional

# Create the MCP server instance
mcp = FastMCP('SalesMCP')

# ■■ Tool 1: Company Lookup ■■
@mcp.tool()
def lookup_company(name: str) -> dict:
    '''Look up company information for sales outreach.'',
    # In production: call LinkedIn API, Clearbit, etc.
    return {
        'name': name,
        'industry': 'SaaS',
        'size': '200 employees',
        'revenue': '$15M ARR',
        'pain_point': 'Manual data workflows',
    }

# ■■ Tool 2: CRM Record Fetch ■■
@mcp.tool()
def get_crm_contact(company: str) -> dict:
    '''Retrieve the primary contact from CRM for a company.'',
    # In production: call Salesforce, HubSpot API
    return {
        'name': 'Sarah Chen',
        'title': 'VP of Engineering',
        'email': f'schen@{company.lower()}.com',
        'last_contact': '2024-01-10',
    }
```

```
# ■■ Start the server (Stdio transport) ■■  
  
if __name__ == '__main__':  
    mcp.run() # listens on stdin/stdout
```

Line-by-Line Breakdown

from mcp.server.fastmcp import FastMCP — FastMCP is a high-level, decorator-based framework for building MCP servers quickly. It abstracts away the protocol details (JSON-RPC message encoding, capability negotiation, error handling) so you focus purely on the tool logic.

mcp = FastMCP('SalesMCP') — Creates an MCP server instance with the name 'SalesMCP'. This name appears when an agent connects and lists available servers. Choose descriptive names — they help orchestrators understand what the server does.

@mcp.tool() decorator — The MCP equivalent of the Agents SDK's @tool. It registers the function as a callable tool in the MCP server's capability registry. The docstring becomes the tool's description — this is what the LLM reads to decide when and how to call it. Type annotations define the JSON Schema for arguments.

mcp.run() — Starts the MCP server in Stdio mode: it reads JSON-RPC requests from stdin and writes responses to stdout. When your agent uses MCPServerStdio, it spawns this script as a subprocess and communicates through those pipes.

■ **Key Insight:** The server code has zero knowledge of which agent or LLM will call it. It just exposes tools via the MCP protocol. Tomorrow, you could swap the OpenAI Agents SDK client for a LangGraph agent or Claude Desktop — your MCP server needs zero changes.

Cell [3]

Connecting an Agent to Your MCP Server

Purpose

Connect the custom MCP server from Cell 2 to an OpenAI agent, demonstrating how the agent auto-discovers and uses MCP tools transparently.

Cell [3] — Agent + MCP Server

```
import asyncio

from agents import Agent, Runner
from agents.mcp import MCPServerStdio
from dotenv import load_dotenv

load_dotenv(override=True)

async def main():
    # Connect to your custom MCP server via Stdio
    async with MCPServerStdio(
        params={
            'command': 'python',
            'args': ['my_mcp_server.py'],
        }
    ) as sales_mcp:

        # Agent auto-discovers tools from the MCP server
        agent = Agent(
            name='SDR_Agent',
            instructions=(
                'You are a sales development rep. '
                'Use your tools to research companies and find contacts '
                'before drafting outreach strategies.'
            ),
            model='gpt-4o-mini',
            mcp_servers=[sales_mcp], # attach MCP server
        )

        result = await Runner.run(
            agent=agent,
            input='Research TechFlow Inc and find their main contact.',
        )

        print(result.final_output)
```

```
asyncio.run(main())
```

■ OUTPUT / CONSOLE

TechFlow Inc. Research Summary:

Industry: SaaS | Size: 200 employees | ARR: \$15M

Pain Point: Manual data workflows

Primary Contact:

Sarah Chen — VP of Engineering

Email: schen@techflow.com | Last Contact: 2024-01-10

Recommendation: Lead with data workflow automation angle.

Sarah is technical — emphasise integration depth and API quality.

Line-by-Line Breakdown

async with MCPServerStdio(params={...}) as sales_mcp: — The async context manager spawns the MCP server subprocess (python my_mcp_server.py), establishes the Stdio connection, performs MCP capability negotiation, and makes the connection available as sales_mcp. On exit, it gracefully shuts down the subprocess.

params={'command': 'python', 'args': ['my_mcp_server.py']} — Tells the MCPServerStdio class how to launch the MCP server process. For a Node.js MCP server, you'd use 'command': 'npx' with appropriate args. The client is agnostic to the server's implementation language.

mcp_servers=[sales_mcp] — This is the key parameter. Instead of tools=[], you pass mcp_servers=[]. The SDK automatically queries the MCP server for its list of tools, converts them into agent-compatible tool definitions, and injects them into the agent's context. You never manually list the tools.

Agent uses tools transparently — From the agent's LLM perspective, MCP tools look identical to direct @tool functions. The LLM sees tool names and descriptions, decides to call them, generates arguments, and receives results. It has no knowledge of whether the tool is local or remote.

■ **Key Insight:** The agent is created **INSIDE** the async with block, because sales_mcp must be active when the agent runs. This is a common gotcha — if you create the agent before the MCP connection is established, it won't have the tools available.

Cell [4]

Using a Pre-built MCP Server — Playwright Browser Agent

Purpose

Connect to the production-ready Playwright MCP server (a Node.js package) to give an agent live browser automation capabilities — navigating websites, clicking elements, and extracting content — without writing a single tool function.

Cell [4] — Playwright MCP Server

```
import asyncio

from agents import Agent, Runner
from agents.mcp import MCPServerStdio
from dotenv import load_dotenv

load_dotenv(override=True)

async def main():
    # Connect to Playwright MCP server (Node.js process)
    async with MCPServerStdio(
        params={
            'command': 'npx',
            'args': ['@playwright/mcp'],
        }
    ) as playwright_mcp:

        # Agent gains full browser automation capabilities
        research_agent = Agent(
            name='Web_Researcher',
            instructions=(
                'You are a web researcher. Use your browser tools to '
                'navigate websites, extract information, and summarise findings. '
                'Always take a screenshot after navigating to verify you are on the right page.'
            ),
            model='gpt-4o',
            mcp_servers=[playwright_mcp],
        )

        result = await Runner.run(
            agent=research_agent,
            input=(
                'Go to news.ycombinator.com and find the top 3 AI-related posts today. '
```

```
'Return their titles and URLs.'
),
)
print(result.final_output)

asyncio.run(main())
```

■ OUTPUT / CONSOLE

Top 3 AI-Related Posts on Hacker News Today:

1. 'Anthropic releases Claude 3.5 Haiku with extended context'

URL: <https://news.ycombinator.com/item?id=42001234>

2. 'OpenAI announces GPT-5 technical report'

URL: <https://news.ycombinator.com/item?id=42001456>

3. 'LLM inference cost drops 90% in 18 months – analysis'

URL: <https://news.ycombinator.com/item?id=42001789>

Line-by-Line Breakdown

'command': 'npx', 'args': ['@playwright/mcp'] — Instead of running a Python script, this launches the Playwright MCP server as a Node.js npx process. The MCPServerStdio class is completely agnostic to this — it just needs a command that speaks MCP protocol over stdin/stdout. This demonstrates MCP's language-independence in action.

Agent auto-discovers Playwright tools — On connection, Playwright MCP exposes dozens of tools: `browser_navigate`, `browser_click`, `browser_type`, `browser_screenshot`, `browser_get_text`, `browser_wait_for_element`, and more. The agent receives all of these automatically via MCP's capability negotiation, with no manual configuration.

model='gpt-4o' — Browser automation tasks are cognitively complex: the agent must reason about page structure, decide which elements to interact with, interpret screenshots, and handle dynamic content. GPT-4o's stronger visual and reasoning capabilities make it a better fit here than gpt-4o-mini.

'Always take a screenshot after navigating' — This instruction in the system prompt demonstrates the Context Over Memory principle: by telling the agent to verify its state with a screenshot, you give it real-time visual context that dramatically improves navigation accuracy without any memory infrastructure.

■ **Key Insight: You got a full browser-automation agent in ~30 lines of code — zero Playwright API knowledge required. This is the power of the MCP ecosystem: the Playwright team built the server once, and every MCP-compatible agent in the world can now browse the web. This is the N+M vs N×M advantage playing out in practice.**

Purpose

Demonstrate connecting to a remote (cloud-hosted) MCP server using the SSE transport, enabling agents to access centralised, team-shared tool infrastructure.

Cell [5] — Remote MCP via SSE & Multi-Server Agents

```
import asyncio

from agents import Agent, Runner
from agents.mcp import MCPServerSse
from dotenv import load_dotenv

load_dotenv(override=True)

async def main():
    # Connect to a REMOTE MCP server via HTTP/SSE
    async with MCPServerSse(
        params={
            'url': 'https://mcp.your-company.com/sales-tools/sse',
            'headers': {
                'Authorization': 'Bearer YOUR_API_KEY',
            },
        }
    ) as remote_mcp:

        agent = Agent(
            name='Enterprise_Agent',
            instructions='You have access to enterprise sales and CRM tools.',
            model='gpt-4o-mini',
            mcp_servers=[remote_mcp],
        )

        result = await Runner.run(
            agent=agent,
            input='Pull the Q4 pipeline report from the CRM.',
        )
        print(result.final_output)

    # Multiple MCP servers simultaneously
    async with (
```

```
MCPServerStdio(params={'command':'python','args':['local_tools.py']}) as local,
MCPServerSse(params={'url':'https://mcp.company.com/sse'}) as remote,
):
agent = Agent(
name='Hybrid_Agent',
instructions='Use both local and remote tools as needed.',
model='gpt-4o-mini',
mcp_servers=[local, remote], # combine both!
)

asyncio.run(main())
```

Line-by-Line Breakdown

MCPServerSse(params={'url': ..., 'headers': ...}) — Establishes an HTTP connection to a remote MCP server. The 'url' points to the server's SSE endpoint. 'headers' passes authentication credentials. Unlike Stdio (which spawns a process), SSE connects over the network — the server can be anywhere in the world.

Authorization: Bearer YOUR_API_KEY — Remote MCP servers must be secured. The MCP protocol itself doesn't mandate an auth scheme, but HTTP bearer tokens are the standard pattern. In production, store your API key in .env and load it with python-dotenv — never hardcode credentials.

mcp_servers=[local, remote] — An agent can connect to multiple MCP servers simultaneously. The SDK merges the tool lists from all connected servers and presents the combined set to the LLM. The agent seamlessly switches between local tools (fast, no network overhead) and remote tools (shared, more powerful) in a single run.

■ **Key Insight:** The multi-server pattern is how enterprise agentic platforms are built. Your company's AI platform might have: one MCP server for CRM tools, one for HR systems, one for financial data, and one for internal knowledge bases. A single agent can access all of them simultaneously — and every team in the company benefits from the shared MCP infrastructure.

Purpose

Build a complete deep research agent that combines Playwright MCP (web browsing), a custom MCP server (data processing), and the Orchestrator+Specialist pattern to produce a structured research report — demonstrating the full MCP ecosystem.

Cell [6] — Deep Research Agent with MCP

```
import asyncio

from agents import Agent, Runner, handoff
from agents.mcp import MCPServerStdio
from pydantic import BaseModel
from typing import List
from dotenv import load_dotenv

load_dotenv(override=True)

class ResearchReport(BaseModel):
    topic: str
    key_findings: List[str]
    sources: List[str]
    summary: str
    recommended_actions: List[str]

async def main():
    async with (
        MCPServerStdio(params={'command': 'npx', 'args': ['@playwright/mcp']}) as browser,
        MCPServerStdio(params={'command': 'python', 'args': ['data_mcp.py']}) as data_tools,
    ):
        # Specialist: Web Research
        web_researcher = Agent(
            name='Web_Researcher',
            instructions='Search the web and gather information on the given topic.',
            model='gpt-4o',
            mcp_servers=[browser],
        )

        # Specialist: Report Writer
        writer = Agent(
            name='Report_Writer',
```

```

instructions='Synthesise research findings into a structured report.',
model='gpt-4o-mini',
mcp_servers=[data_tools],
output_type=ResearchReport,
)

# Orchestrator
orchestrator = Agent(
name='Research_Orchestrator',
instructions=(
'Coordinate deep research: '
'1) Hand off to Web_Researcher to gather info. '
'2) Hand off to Report_Writer to produce the final report.'
),
model='gpt-4o-mini',
handoffs=[handoff(web_researcher), handoff(writer)],
)

result = await Runner.run(
agent=orchestrator,
input='Research the current state of MCP adoption in enterprise AI.',
)

report = result.final_output
print(f'Topic: {report.topic}')
print(f'Findings: {len(report.key_findings)} items')
print(f'Summary: {report.summary}')

asyncio.run(main())

```

■ OUTPUT / CONSOLE

```

Topic: MCP Adoption in Enterprise AI (2024-2025)

Findings: 5 items

Summary: MCP has seen rapid adoption since its Nov 2024 release.
Major IDEs (Cursor, VS Code), AI platforms (Claude, Copilot),
and 500+ community MCP servers now exist. Enterprise adoption
accelerating in Q1 2025, driven by Salesforce & SAP integrations.

```

Line-by-Line Breakdown

Two simultaneous MCP servers (browser + data_tools) — The `async with block` opens both connections in parallel. `web_researcher` gets browser tools; `writer` gets data processing tools. Each specialist agent only has

access to the MCP server it needs — this is the principle of least privilege for agent tool access.

gpt-4o for web_researcher, gpt-4o-mini for writer — Browser automation requires visual reasoning and complex decision-making (gpt-4o). Report synthesis is a straightforward writing task (gpt-4o-mini is sufficient and 10x cheaper). This is the multi-model cost optimisation strategy applied correctly.

output_type=ResearchReport on the writer — The final deliverable is a structured Pydantic object. The orchestrator's result.final_output is already parsed and validated — report.key_findings is a Python list you can iterate, pass to downstream systems, or store in a database immediately.

The complete pipeline — Orchestrator → Web_Researcher (browses web with Playwright MCP) → Report_Writer (formats with data tools MCP) → Structured output. This is a production-grade research automation system in under 60 lines of code.

■ **Key Insight: This pattern — MCP + Handoffs + Structured Output + Multi-Model — is the architecture of real enterprise AI products being shipped today. The MCP layer makes it possible to swap any component (browser tool, data processor, LLM) without touching the orchestration logic. That modularity is what makes the system maintainable at production scale.**

CHAPTER SUMMARY

Concept	Key Takeaway
What is MCP	Open standard for agent-tool communication — the USB-C port for AI applications
USB-C Analogy	N+M components vs N×M integrations — build once, connect everywhere
Three Components	Host (LLM app) → Client (inside host) → MCP Server (tools/resources/prompts)
Stdio Transport	Local subprocess via stdin/stdout — ideal for local tools like filesystem or Playwright
SSE Transport	Remote HTTP server — ideal for cloud APIs, shared enterprise tool infrastructure
Distributed Runtime	Host Service + Worker Runtimes + Gremlin = production-scale multi-agent systems
MCP vs @tool	@tool for single-use local logic; MCP server for shared, reusable, cross-team capabilities
Framework Choice	Choose by team skill & problem fit, not hype. All major frameworks now support MCP.
4 Principles	Metric → Workflow over Autonomy → Bottom-Up → Context over Memory
Debugging	Better context first, then traces, then scientific hypothesis testing